Lecture 06.10: Working with the JSON Module in Python

Python 3.10+ | Jupyter Notebook | PDF notes.pdf

Language / भाषा: English | हिंदी / Hinglish | ← Back to Lecture 06 Module

---

1. Overview & Conceptual Foundations

Learn JSON (JavaScript Object Notation) data serialization and deserialization in Python. Master the 4 core functions: json.dumps() (Python to JSON string), json.loads() (JSON string to Python object), json.dump() (write JSON to file stream), and json.load() (read JSON from file stream), along with formatting using indent=4.

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

Data serialization is an isomorphic mapping between memory-resident object graphs and serialized byte sequences:

$$\mathcal{S} : \text{PyObject} \longrightarrow \text{JSON_String}, \qquad \mathcal{S}^{-1} : \text{JSON_String} \longrightarrow \text{PyObject}$$

Mapping invariants preserve type equivalences:

$$\text{dict} \mapsto \text{Object}, \quad \text{list} \mapsto \text{Array}, \quad \text{bool} \mapsto \text{true/false}, \quad \text{None} \mapsto \text{null}$$

---

4. Structured Implementation & Execution

```
import json
import os

def demonstrate_json_serialization_workflow(filepath: str, initial_data: dict, new
    """
    Demonstrates the complete JSON lifecycle: serialization to string, writing to
    reading back from disk, updating data in-place, and re-saving.
    (Covers Assignment 5 Question 4 workflow).

    Parameters:
    -----------
    filepath : str
        Target JSON file path.
    initial_data : dict
        Initial dictionary (e.g. city populations).
    new_entry_key : str
        New city name to insert.
    new_entry_val : int
        New city population.

    Returns:
    --------
    dict
        Verification of serialized strings and updated loaded dictionaries.
    """
    # 1. dumps: Python dict to JSON String
    json_string = json.dumps(initial_data)

    # 2. dump: Save to file with indent=4 for human readability
    with open(filepath, "w", encoding="utf-8") as f:
        json.dump(initial_data, f, indent=4)

    # 3. load: Read from file back into Python dictionary
    with open(filepath, "r", encoding="utf-8") as f:
        loaded_data = json.load(f)

    # 4. In-place update: Insert new city population
    loaded_data[new_entry_key] = new_entry_val

    # 5. Re-save updated dictionary back to JSON file
    with open(filepath, "w", encoding="utf-8") as f:
        json.dump(loaded_data, f, indent=4)

    # 6. Verify final file contents
    with open(filepath, "r", encoding="utf-8") as f:
        final_loaded_data = json.load(f)

    return {
        "serialized_json_string": json_string,
```

```
        "initial_keys_count": len(initial_data),
        "updated_keys_count": len(final_loaded_data),
        "final_json_data": final_loaded_data,
        "new_entry_confirmed": new_entry_key in final_loaded_data
    }

# Driver test (Assignment 5 Q4: Cities & Populations)
cities_dict = {
    "Delhi": 33000000,
    "Mumbai": 21000000,
    "Bengaluru": 13500000
}
target_json = "cities.json"

json_audit = demonstrate_json_serialization_workflow(target_json, cities_dict, "Hy
print("JSON Serialization & Disk I/O Workflow Audit:")
for k, v in json_audit.items():
    print(f"  {k}: {v}")

# Cleanup
if os.path.exists(target_json):
    os.remove(target_json)
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, docstrings, and verified test assertions.
- **Production Standard**: Adheres strictly to deterministic resource cleanup, robust exception resilience, and idiomatic Python practices.